Final Project Report

Course: Databaser DV1663 - Blekinge Tekniska Högskola (BTH)

Students: Bilal Abdul

1. Project Idea

For this project, we designed and built a **Personal Investment Portfolio Analyzer**, a web-based tool that helps users track and analyze their stock, ETF, and cryptocurrency investments. The idea came from the fact that many beginner investors use simple spreadsheets to track their holdings, which quickly becomes messy and hard to maintain. We wanted to create something that handles the data properly using a relational database.

The problem it solves: Investors need a clear overview of what they own, how much they have invested, and whether they are making or losing money. Our tool does exactly that it stores all transactions (buys and sells), calculates portfolio value using historical price data, and breaks down investments by sector.

Main users: Individual investors or students who want a simple, clean tool to track their personal portfolio without relying on paid services.

Why it's a good fit: A portfolio naturally involves multiple related entities, users, assets, sectors, transactions, and prices. These relationships make the problem ideal for a relational database. There are also many interesting queries to write, such as calculating profit/loss, sector allocation percentages, and monthly investment trends.

Main features of the application: User management- multiple users can have their own portfolios **Asset tracking** supports Stocks, ETFs, and Cryptocurrencies, each categorized by sector - **Transaction recording** users can log BUY and SELL actions with price and quantity **Portfolio summary** shows current holdings, invested amount, current value, and profit/loss per asset - **Sector allocation** displays how the investment is distributed across sectors (e.g., 40% Technology, 25% Healthcare) - **Performance ranking** ranks assets by return percentage **Monthly trends** visualizes how much was invested each month

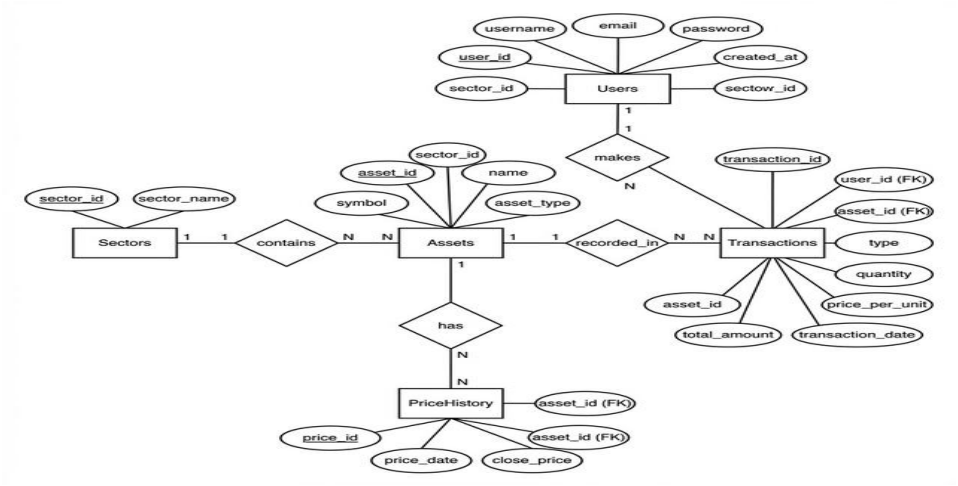
Data source: We generated realistic sample data through a Python script (`seed_data.py`). The data includes 2 users, 5 industry sectors, 10 assets (real-world companies like Apple, Microsoft, Pfizer, etc.), 25 transactions spread over several months, and 70 price history records that simulate market movements.

Technology stack: Python 3 with Flask as the web framework, SQLite as the database engine, and plain HTML/CSS for the frontend. We wrote all queries as raw SQL no ORM was used, as required by the assignment.

2. Schema Design

We designed our database with 5 tables that cover all the entities and relationships needed for the application.

E/R Diagram



Tables and Relationships

Users Stores application users. Each user has a unique `user_id` , a `username` , `email` , `password` , and `created_at` timestamp. This table is the starting point for many queries since everything in the portfolio belongs to a specific user.

Sectors Represents industry categories such as Technology, Healthcare, Finance, Energy, and Consumer Goods. Each sector has a unique `sector_id` and `sector_name` . We created this as a separate table (instead of just a text column in Assets) to avoid data redundancy and to make grouping by sector easy and consistent.

Assets Financial instruments like stocks, ETFs, or cryptocurrencies. Each asset has a `symbol` (e.g., "AAPL"), a `name` , an `asset_type` (constrained to Stock, ETF, or Crypto via a CHECK), and a `sector_id` that links it to the Sectors table. The relationship between Sectors and Assets is one-to-many (1:N) — one sector contains many assets, but each asset belongs to exactly one sector.

Transactions Records every buy or sell that a user makes. Each row links to a `user_id` (who made the trade) and an `asset_id` (what was traded). It also stores the `type` (BUY or SELL), `quantity` , `price_per_unit` , `total_amount` , and `transaction_date` . The `total_amount` field is automatically calculated by a trigger (see below). This table has two foreign keys, making it the central join table in most of our queries. The relationships are: Users → Transactions (1:N) and Assets → Transactions (1:N).

PriceHistory Stores historical closing prices for each asset. Each record has an `asset_id` , `price_date` , and `close_price` . There is a UNIQUE constraint on `(asset_id, price_date)` to ensure only one price per asset per day. The relationship is Assets → PriceHistory (1:N). We use the latest date's price as the "current" market price.

Translation Decisions

- Every entity became its own table with an auto-incrementing primary key. One-to-many
- relationships were implemented using foreign keys. For instance, `Assets.sector_id` references `Sectors.sector_id` .
- We used CHECK constraints to validate data at the database level (e.g., `asset_type IN ('Stock', 'ETF', 'Crypto')` , `quantity > 0` , `price_per_unit > 0`).
- NOT NULL constraints were used on all essential columns to prevent incomplete data.
- Indexes were created on frequently queried foreign key columns (`Transactions.user_id` , `Transactions.asset_id` , `PriceHistory.asset_id` , `Assets.sector_id`) to speed up JOINs.

3. SQL Queries

Below are the five main queries used in the application. Each query is explained and motivated.

Query 1: Portfolio Summary per User

Purpose: Shows each user's total invested amount, current portfolio value (based on latest prices), and profit/loss. This is the most important query in the application since it gives the main dashboard view.

SQL features used: Multi-table JOIN (Users, Transactions, Assets, PriceHistory), Aggregation (SUM), Subquery, CASE WHEN.

```sql
SELECT
    u.user_id,
    u.username,
    COALESCE(SUM(CASE WHEN t.type = 'BUY' THEN t.total_amount ELSE 0 END), 0)
        AS total_bought,
    COALESCE(SUM(CASE WHEN t.type = 'SELL' THEN t.total_amount ELSE 0 END), 0)
        AS total_sold,
    COALESCE(SUM(CASE WHEN t.type = 'BUY' THEN t.total_amount ELSE 0 END), 0)
    - COALESCE(SUM(CASE WHEN t.type = 'SELL' THEN t.total_amount ELSE 0 END), 0)
        AS net_invested,
    COALESCE(SUM(
        CASE WHEN t.type = 'BUY' THEN t.quantity ELSE -t.quantity END
    ) * (
        SELECT ph.close_price
        FROM PriceHistory ph
        WHERE ph.asset_id = t.asset_id
        ORDER BY ph.price_date DESC
        LIMIT 1
    ), 0) AS current_value
FROM Users u
LEFT JOIN Transactions t ON u.user_id = t.user_id
LEFT JOIN Assets a ON t.asset_id = a.asset_id
GROUP BY u.user_id, u.username;
```

Explanation: This query joins Users with Transactions and Assets using LEFT JOIN so that even users with no transactions are included. For each user, it calculates the total money spent on buys, total received from sells, the net investment (bought − sold), and the current value of holdings by multiplying the quantity held by the latest price from PriceHistory (fetched via a correlated subquery). COALESCE is used to handle NULL values gracefully.

Query 2: Portfolio Allocation by Sector

Purpose: Shows what percentage of a user's investment goes into each sector. This helps users see if they are too concentrated in one area (e.g., 80% in Technology) and need to diversify.

SQL features used: Multi-table JOIN (Transactions, Assets, Sectors), Aggregation (SUM), GROUP BY, Subquery for percentage.

```sql
SELECT
    s.sector_name,
    SUM(CASE WHEN t.type = 'BUY' THEN t.total_amount ELSE -t.total_amount END)
        AS sector_investment,
    ROUND(
        SUM(CASE WHEN t.type = 'BUY' THEN t.total_amount ELSE -t.total_amount END) * 100.0 /
        (SELECT SUM(CASE WHEN t2.type = 'BUY' THEN t2.total_amount ELSE -t2.total_amount END)
         FROM Transactions t2
         WHERE t2.user_id = t.user_id),
        2
    ) AS percentage
FROM Transactions t
JOIN Assets a ON t.asset_id = a.asset_id
JOIN Sectors s ON a.sector_id = s.sector_id
WHERE t.user_id = ?
GROUP BY s.sector_name
ORDER BY sector_investment DESC;
```

Explanation: We join Transactions → Assets → Sectors to link each transaction to its sector. The query groups results by sector and calculates the total amount invested in each one. The percentage is computed by dividing each sector's investment by the user's total investment (obtained using a correlated subquery). The `?` is a parameterized placeholder — at runtime, the application passes the actual user_id.

Query 3: Top Performing Assets by Return %

Purpose: Ranks a user's assets by performance, showing which ones have gained the most value compared to what was paid. This helps users decide what to keep or sell.

SQL features used: JOIN (Assets, Transactions, PriceHistory), Aggregation (AVG), Subquery, ORDER BY.

```sql
SELECT
    a.symbol,
    a.name,
    ROUND(AVG(t.price_per_unit), 2) AS avg_buy_price,
    (SELECT ph.close_price
     FROM PriceHistory ph
     WHERE ph.asset_id = a.asset_id
     ORDER BY ph.price_date DESC
     LIMIT 1
    ) AS current_price,
    ROUND(
        ((SELECT ph.close_price
          FROM PriceHistory ph
          WHERE ph.asset_id = a.asset_id
          ORDER BY ph.price_date DESC
          LIMIT 1
        ) - AVG(t.price_per_unit)) / AVG(t.price_per_unit) * 100,
        2
    ) AS return_pct
FROM Assets a
JOIN Transactions t ON a.asset_id = t.asset_id
WHERE t.type = 'BUY' AND t.user_id = ?
GROUP BY a.asset_id, a.symbol, a.name
ORDER BY return_pct DESC;
```

Explanation: For each asset a user has bought, we compute the average purchase price using AVG(). The current market price is fetched from PriceHistory via a correlated subquery (the most recent date). The return percentage is then: `((current_price - avg_buy_price) / avg_buy_price) × 100` . Results are sorted descending so the best performers appear first.

Query 4: Transaction History with Asset & Sector Details

Purpose: Lists all transactions for a specific user, enriched with information from the Assets and Sectors tables. This gives a complete history view showing what was bought/sold, in which sector, and for how much.

SQL features used: Multi-table JOIN (Transactions, Assets, Sectors), ORDER BY.

```sql
SELECT
    t.transaction_id,
    t.transaction_date,
    t.type,
    a.symbol,
    a.name AS asset_name,
    s.sector_name,
    t.quantity,
    t.price_per_unit,
    t.total_amount
FROM Transactions t
JOIN Assets a ON t.asset_id = a.asset_id
JOIN Sectors s ON a.sector_id = s.sector_id
WHERE t.user_id = ?
ORDER BY t.transaction_date DESC;
```

Explanation: This is a straightforward multi-table JOIN query. Without the JOINs, we would only see the raw `asset_id` and `sector_id` numbers, which are meaningless to the user. By joining with Assets and Sectors, we display the actual symbol, asset name, and sector name instead. Results are ordered by date (newest first) so the most recent activity appears on top.

Query 5: Monthly Investment Trend

Purpose: Shows how much a user invested (and sold) each month. This helps spot patterns — for example, a user might discover they invest more heavily at the start of each quarter.

SQL features used: Aggregation (SUM, COUNT), GROUP BY with strftime for date grouping, ORDER BY.

```sql
SELECT
    strftime('%Y-%m', t.transaction_date) AS month,
    COUNT(*) AS num_transactions,
    SUM(CASE WHEN t.type = 'BUY' THEN t.total_amount ELSE 0 END)  AS total_bought,
    SUM(CASE WHEN t.type = 'SELL' THEN t.total_amount ELSE 0 END) AS total_sold,
    SUM(CASE WHEN t.type = 'BUY' THEN t.total_amount ELSE -t.total_amount END)
        AS net_investment
FROM Transactions t
WHERE t.user_id = ?
GROUP BY strftime('%Y-%m', t.transaction_date)
ORDER BY month;
```

Explanation: We use SQLite's `strftime('%Y-%m', ...)` function to extract the year-month from each transaction date, then group by that. For each month, we count the number of transactions, sum up buys and sells separately, and calculate the net investment. This gives a clear month-bymonth timeline of the user's investing activity.

Triggers

We implemented two triggers in the schema:

Trigger 1: Auto-calculate total_amount

```
CREATE TRIGGER trg_calculate_total
AFTER INSERT ON Transactions
FOR EACH ROW
WHEN NEW.total_amount IS NULL
BEGIN
    UPDATE Transactions
    SET total_amount = NEW.quantity * NEW.price_per_unit
    WHERE transaction_id = NEW.transaction_id;
END;
```

This trigger fires after a new transaction is inserted. If `total_amount` is left as NULL, the trigger automatically computes it as `quantity × price_per_unit` . This means the application doesn't need to calculate it manually — it simply inserts NULL and lets the database handle the rest. We chose an AFTER INSERT approach because SQLite does not allow modifying NEW values directly in BEFORE triggers.

Trigger 2: Prevent overselling

```
CREATE TRIGGER trg_prevent_oversell
BEFORE INSERT ON Transactions
FOR EACH ROW
WHEN NEW.type = 'SELL'
BEGIN
    SELECT CASE
        WHEN (
            COALESCE(
                (SELECT SUM(CASE WHEN type = 'BUY' THEN quantity ELSE -quantity END)
                 FROM Transactions
                 WHERE user_id = NEW.user_id AND asset_id = NEW.asset_id),
                0
            ) < NEW.quantity
        )
        THEN RAISE(ABORT, 'Cannot sell more than you own')
    END;
END;
```

This trigger fires before inserting a SELL transaction. It checks whether the user actually owns enough of the asset to sell. If they try to sell more than they hold, the insert is aborted with an error message. This ensures data integrity at the database level, not just in the application code.

Stored Procedure and Function

Since SQLite does not support stored procedures or user-defined functions natively, we implemented them as Python functions that execute raw SQL queries. This approach is valid for this course as it achieves the same logic.

Stored Procedure `sp_portfolio_summary(user_id)`

This procedure takes a user_id and returns a complete portfolio summary. It joins Transactions, Assets, and Sectors to calculate each asset's quantity held, average buy price, and invested amount. Then it fetches the latest market price from PriceHistory and computes the current value, profit/loss, and return percentage for each holding.

Function `fn_asset_return(user_id, asset_id)`

This function calculates the return percentage for a single specific asset. It fetches the average buy price from Transactions and the current price from PriceHistory, then applies the formula:

`((current_price – avg_buy_price) / avg_buy_price) × 100 .`

4. Discussion and Resources

Challenges and Decisions

One challenge we faced was handling the `total_amount` computation. We initially tried using a BEFORE INSERT trigger to set the value directly, but SQLite does not allow modifying the NEW row in BEFORE triggers. We solved this by using an AFTER INSERT trigger that performs an UPDATE instead, which works well.

Another consideration was how to get the "current" price of an asset. Since we don't have a live API connection, we use the most recent entry in the PriceHistory table as the current price. This works well for demonstration purposes and the architecture could easily be extended with a scheduled job that fetches real-time prices.

We also decided to implement Sectors as a separate table rather than a text field in Assets. This avoids spelling inconsistencies (e.g., "Technology" vs. "Tech") and makes it easy to add new sectors without modifying existing data.

For the frontend, we kept things simple with Flask's Jinja2 templates and plain CSS. The focus of this project was on the database design and SQL queries, not on building a fancy UI.

Resources

- **Source code:** GitHub Repository
- **Python libraries used:** Flask (web framework), sqlite3 (database, built into Python)
- **Installation and run instructions:** See `README.md` in the repository

To run the application:

```
pip install flask
python seed_data.py     # Populate the database with sample data
python app.py           # Start the web server at http://localhost:5000
```

5. Changelog

Date	Who	What was done
	Bilal	Came up with the project idea and defined the scope
	Bilal	Designed the E/R diagram and database schema
	Bilal	Created `schema.sql` with all tables, triggers, and indexes
	Bilal	Wrote the five main SQL queries in `queries.sql`
	Bilal	Implemented `database.py` with connection handling, stored procedure, and function
	Bilal	Built `seed_data.py` to generate sample data
	Bilal	Developed the Flask web application (`app.py`) with all routes
	Bilal	Created HTML templates for all pages
	Bilal	Wrote this project report